Fatih Mehmet Serezli 220104004012

1)

a) $2^{2n}+1=O(2^n) \Rightarrow \begin{matrix} f(n)=2^{2n}+1 \\ g(n)=2^n \end{matrix} \Rightarrow \lim_{n\to\infty} \frac{f(n)}{g(n)} = \lim_{n\to\infty} \frac{2^{2n}+1}{2^n} = \lim_{n\to\infty} \left(\underbrace{\frac{2^{2n}}{2^n}}_{2^n} + \underbrace{\frac{1}{2^n}}_{0}\right)$

$= \lim_{n\to\infty} 2^n = \infty \Rightarrow$ False, $f(n)$ grows faster than $g(n)$

b) $n^3-4n^2+7=\Theta(2^n) \Rightarrow \begin{matrix} f(n)=n^3-4n^2+7 \\ g(n)=2^n \end{matrix} \Rightarrow \lim_{n\to\infty} \frac{f(n)}{g(n)} = \lim_{n\to\infty} \left(\underbrace{\frac{n^3}{2^n}}_{0} - \underbrace{\frac{4n^2}{2^n}}_{0} + \underbrace{\frac{7}{2^n}}_{0}\right)$

$= \lim_{n\to\infty} 0 = 0 \Rightarrow$ False, $f(n)$ grows slower than $g(n)$

c) $n^3(1+\sqrt{n})=O(n^3\log n) \Rightarrow \begin{matrix} f(n)=n^3(1+\sqrt{n}) \\ g(n)=n^3\log n \end{matrix} \Rightarrow \lim_{n\to\infty} \frac{f(n)}{g(n)} = \lim_{n\to\infty} \frac{\cancel{n^3}(1+\sqrt{n})}{\cancel{n^3}\log n}$

$= \lim_{n\to\infty} \left(\underbrace{\frac{1}{\log n}}_{0} + \frac{\sqrt{n}}{\log n}\right) = \lim_{n\to\infty} \frac{\sqrt{n}}{\log n} \xrightarrow[\text{(L'Hospital)}]{} \lim_{n\to\infty} \frac{\frac{1}{2\sqrt{n}}}{\frac{1}{n}}$

$= \lim_{n\to\infty} \frac{n}{2\sqrt{n}} = \lim_{n\to\infty} \frac{\sqrt{n}}{2} = \infty \Rightarrow$ False, $f(n)$ grows faster.

d) $28n=O(7n^2) \Rightarrow \begin{matrix} f(n)=28n \\ g(n)=7n^2 \end{matrix} \Rightarrow \lim_{n\to\infty} \frac{f(n)}{g(n)} = \lim_{n\to\infty} \frac{28n}{7n^2}$

$= \lim_{n\to\infty} \frac{28}{7n} = 0 \Rightarrow$ True, $f(n)$ grows slower than $g(n)$

e) $n+\log n+21=O(7n^2) \Rightarrow \begin{matrix} f(n)=n+\log n+21 \\ g(n)=7n^2 \end{matrix} \Rightarrow \lim_{n\to\infty} \frac{f(n)}{g(n)} = \lim_{n\to\infty} \left(\underbrace{\frac{n}{7n^2}}_{0} + \underbrace{\frac{\log n}{7n^2}}_{0} + \underbrace{\frac{21}{7n^2}}_{0}\right)$

(Proved in c)

$= \lim_{n\to\infty} 0 = 0 \Rightarrow$ True, $f(n)$ grows slower.

f) $n^2+9n-13=\Theta(n^2) \Rightarrow \begin{matrix} f(n)=n^2+9n-13 \\ g(n)=n^2 \end{matrix} \Rightarrow \lim_{n\to\infty} \frac{f(n)}{g(n)} = \lim_{n\to\infty} \left(\underbrace{\frac{n^2}{n^2}}_{1} + \underbrace{\frac{9n}{n^2}}_{0} - \underbrace{\frac{13}{n}}_{0}\right)$

$= \lim_{n\to\infty} 1 = 1 \Rightarrow$ True, $f(n)$ and $g(n)$ has same grows rate.

Fatih Mehmet Severli 220104004012

2) - Compare at $\log n$ and $7n \Rightarrow \lim_{n\to\infty} \frac{7n}{\log n} \xrightarrow[\text{(L'hopital)}]{} \lim_{n\to\infty} \frac{7}{\frac{1}{n}} = \lim_{n\to\infty} 7n = \infty$

$\Downarrow$

$(7n > \log n)$

- Compare at $7n$ and $2n^2 \Rightarrow \lim_{n\to\infty} \frac{2n^2}{7n} = \lim_{n\to\infty} \frac{2n}{7} = \infty \Rightarrow (2n^2 > 7n)$

- Compare at $2n^2$ and $3n^4 \Rightarrow \lim_{n\to\infty} \frac{3n^4}{2n^2} = \lim_{n\to\infty} \frac{3n^2}{2} = \infty \Rightarrow (3n^4 > 2n^2)$

- Compare at $3n^4$ and $3^n \Rightarrow \lim_{n\to\infty} \frac{3^n}{3n^4} = \lim_{n\to\infty} \frac{3^{n-1}}{n^4} = \lim_{n\to\infty} \frac{\ln(3^{n-1})}{\ln(n^4)}$

$= \lim_{n\to\infty} \frac{(n-1)\ln(3)}{4\ln(n)} = \lim_{n\to\infty} \left(\underbrace{\frac{n-1}{\ln(n)}}_{\text{infinity}} \cdot \underbrace{\frac{\ln(3)}{4}}_{\text{constant}} \right) = \infty \Rightarrow (3^n > 3n^4)$

(proved in compare at $\log n$ and $7n$)

- Compare at 3^n and $n! \Rightarrow \lim_{n\to\infty} \frac{n!}{3^n} = \underbrace{\frac{1}{3} \cdot \frac{2}{3} \cdot \frac{3}{3} \cdot \frac{4}{3} \cdot \frac{5}{3}}_{\text{constants}} \cdots \underbrace{\frac{n-1}{3} \cdot \frac{n}{3}}_{\text{infinity (it n goes to } \infty)} = \infty \; (n! > 3^n)$

$$\boxed{n! > 3^n > 3n^4 > 2n^2 > 7n > \log n}$$

3) a)
```
int sum=0;
for(int i=0; i<a; i++) // loop body executes a times
    for(int j=0; j<b; j++) // loop body executes b times and the loop executes a
                              times. So the total execute count at the inner loop's
                              body is a.b times. -> O(a.b)
```

b)
```
int sum=0;
for(int i=0; i<a; ++i)
    sum+=i;
    i=i*2; //Since i doubles at each step we can express it as i=2^k.
              when 2^k=a, the amount at execute time the loop is: k=log_2 a
```

$\downarrow$

$\boxed{O(\log a)}$

c)
```
for(int i=0; i<n; i++) // body executes n times
    for(int j=0; j<n; j++) //this loop executes n times and its body executes n times
                             for each. So total count is n^2
    for(int k=0; k<n; k++) // this loop executes n^2 times and its body executes
                              n times for each. So total count is n^3
```

$\downarrow$

$\boxed{O(n^3)}$

Fatih Mehmet Serezli 220104004012

4)
```
int Lowest_Floor(int n, Toy toy) { /* n represents the floors of the building.
                                     and expected to be 100, but doesn't matter what it is.
                                     Toy represents the toy. I am using it as an object and
                                     assume that it has "boolean doesBreak(int floor)" method. */
    int bot = 0;
    int top = n;
    while (top - bot > 1) {
        int mid = (bot + top) / 2;
        if (toy.doesBreak(mid)) top = mid;
        else bot = mid;
    }
    if (toy.doesBreak(bot)) return bot;
    else return top;
}
```

Explanation: My algorithm works like binary search algorithm. It always tries middle of the possible floors and if the toy breaks, it eliminates the higher floors, else it eliminates lower floors. For each step half of the possible floors are eliminated. That's why the complexity of my algorithm is $\log_2 n$. $\rightarrow \Theta(\log n)$